

Backend – Assignment – W2 - Native HTTP Module and Manual Routing

Sat Panha – Group 3 – Software Engineer

Exercise 1 – Analyze

Q1 – What error message do you see in the terminal when you access `http://localhost:3000`? What line of code causes it?

When I run the code and go to `http://localhost:3000`, I get this **error:TypeError: res.endd** is not a function because in the code is `res.endd()`, and it should be `res.end()` at line 5.

Q2 - What is the purpose of `res.write()` and how is it different from `res.end()`?

- `res.write()` sends a part of the response.
 - `res.end()` twll the server to stop when all data have been responded.
-

Q3 -- What do you think will happen if `res.end()` is not called at all ?

If we forget to call `res.end()`, the response never finishes. That means:

- the server could crash
 - Client will wait until timeout because server has not stop to request
 - Server will never got a respond
-

Q4 - Why do we use `http.createServer()` instead of just calling a function directly?

We use `http.createServer()` because it helps:

- Build a basic HTTP server
 - Handle requests and responses easily
 - Take care of things like headers and connections, it mean we don't have to do everything ourselves
-

Q5 - -- How can the server be made more resilient to such errors during development?

Here are a few way to made more resilient to such errors during development:

- using try-catch to catch synchronous errors to handle both side inside the request handler
 - add global listener on the server instance to handle unexpected error
-

Exercise 2 – Manipulate

Q1 - What happens when you visit a URL that doesn't match any of the three defined?

It will show 404 - Page Not Found.

Q2 - Why do we check both the req.url and req.method?

Because the Same URL can handle different operations:

- GET /users : retrieve data
 - POST /users : send data
-

Q3 - What MIME type (Content-Type) do you set when returning HTML instead of plain text?

- For HTML pages: Content-Type: text/html
 - For plain text: Content-Type: text/plain
-

Q4 - How might this routing logic become harder to manage as routes grow?

When we add more routes:

- The code gets long and messy
 - It's harder to find, fix bugs and maintain
 - We repeat things like headers and error handling
 - It's not clear and don't know code handles what
-

Q5 - What benefits might a framework offer to simplify this logic?

Frameworks like **Node**:

- Define routes easily with HTTP methods and paths, without using complex conditionals
 - Easily add common features like authentication, logging, and input validation
 - Save time and reduce bugs
 - Make error handling simpler
-

Exercise 3 – Create

Q1 - Why do we listen for data and end events when handling POST?

Because POST data doesn't come all at once, it comes in chunks.

- data event gives us each chunk
 - end tells us the request is done
-

Q2 - What would happen if we didn't buffer the body correctly?

- We might miss some data
 - The form might be incomplete or broken
 - Big data could crash the server
-

Q3 - What is the format of form submissions when using the default browser form POST?

The default format for browser form POST is application/x-www-form-urlencoded:

- Data is sent as name-value pairs (name=value).
 - Multiple pairs are joined with &.
 - Spaces become +, special characters are percent-encoded.
-

Q4 - Why do we use fs.appendFile instead of fs.writeFile?

- appendFile adds new data without deleting the old one
- writeFile would replace everything, so we'd lose previous messages

Q5 - How could this be improved or made more secure?

- Check and clean user input to block bad data
 - Don't allow too many requests from the same user (spam)
 - Store the data in a database, not a text file
 - Use HTTPS for security
 - Add a CAPTCHA
 - Use tokens to protect from CSRF attacks
-